COMPLETE TECHNICAL INTERVIEW GUIDE

C#, ASP.NET Core & SQL Server Master Questions & Answers

A comprehensive, end-to-end reference handbook containing 91 essential interview questions, detailed conceptual explanations, comparison matrices, and production-ready code examples.

Topics Covered: C# Fundamentals, CLR, Memory Management, ASP.NET Core, CQRS, Microservices, Security, Entity Framework Core, SQL Server Performance Tuning & Indexing

Generated: August 6, 2026

Table of Contents

Section 1: C# & .NET Core Fundamentals

Q1. What is a Sealed Class?

Q2. What are Access Modifiers in C#?

Q3. What is Dependency Injection? Why do we use it?

Q4. What is Middleware?

Q5. What is Routing?

Q6. What are Filters?

Q7. What is an Action Filter?

Q8. How do you handle Global Exception Handling?

Q9. What is async/await? Why is it used?

Q10. Difference between Task and Thread?

Q11. Difference between IEnumerable and IQueryable?

Q12. Difference between Value Type and Reference Type?

Q13. Difference between Struct and Class?

Q14. What are Extension Methods?

Q15. Explain JWT Authentication. Where is the token stored?

Q16. Why is DbContext registered as Scoped?

Q17. Explain your Project Architecture.

Q18. What is Code First? Can we create database tables without SQL scripts?

Q19. Difference between PUT and PATCH?

Q20. Why do we use POST instead of GET?

Q21. Explain the ASP.NET Core Request Life Cycle.

Q22. Explain LINQ and its methods.

Q23. Difference between Eager, Lazy and Explicit Loading?

Q24. Difference between ViewData, ViewBag and TempData?

Q25. Difference between Session and Cookies?

Q26. Difference between readonly, const and static?

Q27. Difference between SOAP and REST API?

Q28. What is CORS?

Q29. Difference between String and StringBuilder?

Q30. How does CLR work?

Q31. Explain SOLID Principles.

Q32. What is API Throttling?

Section 2: ASP.NET Core Advanced, Microservices & Architecture

Q33. Authentication vs Authorization?

Q34. JWT vs OAuth?

Q35. Access Token vs Refresh Token?

Q36. What is CQRS?

Q37. What is MediatR?

Q38. What is Clean Architecture?

Q39. What is Repository Pattern?

Q40. What is Unit of Work?

Q41. What is AutoMapper?

Q42. What is Redis?

Q43. What is SignalR?

Q44. What is Serilog?

Q45. What are Health Checks?

Q46. What is Docker?

Q47. What is Kubernetes?

Q48. RabbitMQ vs Kafka?

Q49. What is gRPC?

Q50. What is API Versioning?

Q51. What is Swagger/OpenAPI?

Q52. What is Idempotency?

Q53. What is CancellationToken?

Q54. What are Minimal APIs?

Q55. What are Background Services?

Q56. Microservices vs Monolithic Architecture?

Q57. Memory Cache vs Distributed Cache?

Q58. What is Response Caching?

Q59. What is Output Caching?

Q60. What is Polly Retry Policy?

Q61. Explain Dependency Injection Lifetimes.

Q62. Explain API Rate Limiting.

Section 3: SQL Server Database Engineering & Optimization

Q63. How do you optimize a Stored Procedure?

Q64. How do you measure Stored Procedure execution time?

Q65. Clustered vs Non-Clustered Index?

Q66. How do Transactions work?

Q67. Function vs Stored Procedure?

Q68. Stored Procedure vs View?

Q69. Find the 5th Highest Salary.

Q70. Find the Top 3 Highest Salaries.

Q71. What are SQL Constraints?

Q72. DELETE vs TRUNCATE vs DROP?

Q73. Why can DELETE be rolled back but TRUNCATE usually cannot?

Q74. What is CTE?

Q75. Temp Table vs Table Variable?

Q76. Find Duplicate Records using CTE.

Q77. ROW_NUMBER vs RANK vs DENSE_RANK?

Q78. EXISTS vs IN?

Q79. INNER JOIN vs LEFT JOIN vs RIGHT JOIN?

Q80. UNION vs UNION ALL?

Q81. What is Normalization?

Q82. What is Denormalization?

Q83. What are ACID Properties?

Q84. What is Deadlock?

Q85. How do you read an Execution Plan?

Q86. What is Index Fragmentation?

Q87. What are Window Functions?

Q88. What is a Cursor?

Q89. How do you improve SQL Query Performance?

Q90. Primary Key vs Unique Key?

Q91. Foreign Key vs Composite Key?

Section 1: C# & .NET Core Fundamentals

Q1. What is a Sealed Class?

A **sealed class** in C# is a class marked with the `sealed` modifier that prevents other classes from inheriting from it. If a class tries to derive from a sealed class, the C# compiler throws a compile-time error.

Why use Sealed Classes?

- **Security & Integrity:** Ensures business logic cannot be altered or overridden by subclassing.
- **Performance Optimization:** Allows the JIT compiler to perform virtual call elimination (devirtualization), improving method call efficiency.

```
// Sealed class example
public sealed class SecurityTokenProvider
{
    public string GenerateToken(string userId)
    {
        return $"TOKEN-{userId}-{Guid.NewGuid()}";
    }
}

// The following will cause a compilation error:
// public class CustomTokenProvider : SecurityTokenProvider { } // Error!
```

Q2. What are Access Modifiers in C#?

Access modifiers define the accessibility/visibility of classes, methods, properties, and variables across assemblies and namespaces.

Modifier	Accessibility Scope
`public`	Accessible from anywhere in the current project or referencing assemblies.
`private`	Accessible only within the declaring class/struct. (Default for class members)
`protected`	Accessible within the declaring class and derived classes.
`internal`	Accessible anywhere within the same assembly (DLL/EXE). (Default for top-level classes)
`protected internal`	Accessible within the same assembly OR from derived classes in other assemblies.
`private protected`	Accessible within the declaring class OR derived classes within the *same* assembly (C# 7.2+).

```
public class BankAccount
{
    private decimal _balance; // Private field
    protected string AccountType; // Accessible in derived classes
    internal string BranchCode; // Accessible in same assembly

    public decimal GetBalance() => _balance; // Public access
}
```

Q3. What is Dependency Injection? Why do we use it?

Dependency Injection (DI) is a software design pattern used to achieve *Inversion of Control (IoC)* between classes and their dependencies. Instead of a class instantiating its own dependencies (using `new`), dependencies are provided ("injected") from an external framework/container (e.g., ASP.NET Core Built-in IoC container).

Why do we use Dependency Injection?

- **Decoupling:** Classes depend on abstractions (interfaces), not concrete implementations (SOLID Dependency Inversion Principle).
- **Testability:** Enables easy mocking of dependencies during unit testing.
- **Maintainability & Flexibility:** Implementations can be swapped easily in DI registration without changing consuming classes.
- **Lifetime Management:** Automates creation, scoping, and disposal of objects.

```
// Abstraction
public interface IEmailService { void Send(string to, string body); }

public class SmtpEmailService : IEmailService
{
    public void Send(string to, string body) ⇒ Console.WriteLine($"Email sent to
{to}");
}

// Injected via Constructor
public class OrderProcessor
{
    private readonly IEmailService _emailService;
    public OrderProcessor(IEmailService emailService) // Injected here
    {
        _emailService = emailService;
    }
    public void ProcessOrder(string userEmail) ⇒ _emailService.Send(userEmail, "Order
Confirmed!");
}
```

Q4. What is Middleware?

Middleware in ASP.NET Core is software assembled into an application pipeline to handle HTTP requests and responses. Each component can perform operations before passing the request to the next middleware (via `RequestDelegate next`) or short-circuit the pipeline altogether.

```
// Custom Inline Middleware
app.Use(async (context, next) ⇒
{
    // 1. Logic BEFORE passing to next middleware
    Console.WriteLine($"Request URL: {context.Request.Path}");

    await next.Invoke(); // Call next middleware

    // 2. Logic AFTER downstream middleware completes
    Console.WriteLine($"Response Status: {context.Response.StatusCode}");
});

// Built-in Middleware pipeline ordering:
// UseExceptionHandler → UseHsts → UseHttpsRedirection → UseStaticFiles →
UseRouting → UseCORS → UseAuthentication → UseAuthorization → MapControllers
```

Q5. What is Routing?

Routing is the mechanism by which ASP.NET Core maps incoming HTTP requests (URLs and HTTP verbs) to specific executable endpoints (Controller Actions or Minimal API handlers).

- **Conventional Routing:** Routes configured centrally in `Program.cs` (e.g., `{controller=Home}/{action=Index}/{id?}`). Common in MVC apps.
- **Attribute Routing:** Routes defined directly on Controllers/Actions using `[Route]`, `[HttpGet]`, etc. Standard for Web APIs.

```
[ApiController]
[Route("api/[controller]")] // Attribute routing: api/products
public class ProductsController : ControllerBase
{
    [HttpGet("{id:int}")] // GET api/products/5
    public IActionResult GetById(int id) ⇒ Ok(new { Id = id, Name = "Laptop" });
}
```

Q6. What are Filters?

Filters in ASP.NET Core allow running code before or after specific stages in the action execution pipeline. They help enforce cross-cutting concerns (logging, authorization, validation, caching) without duplicating code across controllers.

Filter Pipeline Order:

1. **Authorization Filters:** Evaluates if user is authorized. Runs first.
2. **Resource Filters:** Executes after Auth; useful for caching or short-circuiting heavy requests.
3. **Action Filters:** Runs immediately before and after action execution.
4. **Exception Filters:** Handles uncaught exceptions thrown during action execution.
5. **Result Filters:** Runs before and after executing the action result (e.g., view rendering / JSON serialization).

Q7. What is an Action Filter?

An **Action Filter** is a specific type of filter that executes right *before* and *after* a controller action method runs. It can inspect/modify arguments passed into the action or modify the result returned from the action.

```
public class RequestLoggingFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext context)
    {
        // Executes BEFORE controller action
        Console.WriteLine($"Executing action:
{context.ActionDescriptor.DisplayName}");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        // Executes AFTER controller action completes
        Console.WriteLine("Action executed successfully.");
    }
}

// Applied to controller action:
[ServiceFilter(typeof(RequestLoggingFilter))]
[HttpGet]
public IActionResult GetData() ⇒ Ok("Data");
```

Q8. How do you handle Global Exception Handling?

In ASP.NET Core, global exception handling ensures unhandled exceptions are caught gracefully, logged, and formatted into standardized error responses (e.g., ProblemDetails) without exposing stack traces to clients.

Modern approaches include using custom Exception Middleware or .NET 8's `IExceptionHandler` interface.

```
// .NET 8+ IExceptionHandler approach
public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) => _logger =
logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext, Exception exception, CancellationToken
cancellationToken)
    {
        _logger.LogError(exception, "Unhandled Exception occurred: {Message}",
exception.Message);

        var problemDetails = new ProblemDetails
        {
            Status = StatusCodes.Status500InternalServerError,
            Title = "Server Error",
            Detail = exception.Message
        };

        httpContext.Response.StatusCode = problemDetails.Status.Value;
        await httpContext.Response.WriteAsJsonAsync(problemDetails,
cancellationToken);
        return true; // Exception handled
    }
}

// Registration in Program.cs:
// builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
// app.UseExceptionHandler();
```

Q9. What is async/await? Why is it used?

`async` and `await` are C# language keywords used for asynchronous programming. They allow asynchronous code to be written sequentially like synchronous code without blocking the executing thread during long-running I/O operations (database access, file operations, web requests).

> **Why use async/await?**
>
> **Thread Pool Efficiency & Scalability:** While waiting for I/O (e.g., SQL query execution), the executing thread is released back to the ThreadPool to handle other incoming requests instead of sitting idle. This drastically improves backend throughput under high concurrency.

```
public async Task<User> GetUserByIdAsync(int id)
{
    // The thread is yielded while waiting for database response
    var user = await _dbContext.Users.FindAsync(id);
    return user; // Resumed when query completes
}
```

Q10. Difference between Task and Thread?

Feature	Thread	Task (Task Parallel Library)
Level of Abstraction	Low-level OS execution context managed directly by OS/CLR.	High-level abstraction over asynchronous operation.
Resource Cost	Heavy overhead (~1MB stack memory per thread + context switching cost).	Lightweight object created on the heap. Uses ThreadPool threads.
Return Value	Does not easily return a value. Requires complex synchronization.	Can return values easily via `Task<TResult>`.
Composition	Hard to chain or cancel.	Supports `await`, `Task.WhenAll`, `Task.WhenAny`, and cancellation tokens.
Use Case	Long-running background compute tasks bound to CPU.	I/O-bound operations and concurrent async programming.

Q11. Difference between IEnumerable and IQueryable?

Feature	IEnumerable<T>	IQueryable<T>
Namespace	`System.Collections.Generic`	`System.Linq`
Execution Location	In-Memory (Client-side evaluation).	Out-of-Memory (Database / Provider side evaluation).
Query Translation	Uses delegate methods (Funcs). Filters data after loading into RAM.	Uses Expression Trees. Translates LINQ directly into native SQL.
Best Used For	In-memory collections (Lists, Arrays, HashSets).	Querying external data sources (Entity Framework Core, SQL databases).

```
// IEnumerable: Fetches ALL rows from DB first, then filters in RAM!
IEnumerable<User> usersList = _context.Users;
var activeUsers = usersList.Where(u ⇒ u.IsActive); // Transfers whole table over network!

// IQueryable: Generates "SELECT * FROM Users WHERE IsActive = 1" on DB!
IQueryable<User> usersQuery = _context.Users;
var activeUsersQuery = usersQuery.Where(u ⇒ u.IsActive).ToList(); // Executes SQL on DB
```

Q12. Difference between Value Type and Reference Type?

Feature	Value Type	Reference Type
Storage Location	Stored on the **Stack** (or inline in containing object).	Stored on the **Heap**; stack holds memory pointer reference.
Examples	`int` , `float` , `bool` , `char` , `struct` , `enum`	`class` , `string` , `object` , `interface` , `delegate` , `array`
Assignment Behavior	Copies the actual *value*. Changes to copy do not affect original.	Copies the *memory reference*. Both variables point to same heap object.
Default Nullability	Cannot be null (unless declared Nullable: `int?`).	Can be `null` .
Garbage Collection	Cleaned up automatically when thread stack pops frame.	Managed and collected by CLR Garbage Collector (GC).

Q13. Difference between Struct and Class?

Feature	Struct	Class
Type	Value Type (Allocated on Stack).	Reference Type (Allocated on Heap).
Inheritance	Does not support inheritance (cannot derive from struct/class).	Supports single inheritance and polymorphism.
Default Constructor	Cannot have parameterless constructor (prior to C# 10).	Can define parameterless and parameterized constructors.
Nullability	Non-nullable by default.	Nullable reference type.
Use Case	Small, immutable, lightweight data structures (Point, Vector, DateTime).	Complex domain objects with identity, behaviors, and long lifespans.

Q14. What are Extension Methods?

Extension methods allow developers to add new methods to existing types without modifying the original source code, deriving from the type, or recompiling.

Rules for extension methods:

- Must be declared inside a **static class**.
- The method itself must be **static**.
- The first parameter must use the `this` keyword specifying the target type.

```
public static class StringExtensions
{
    // Extension method extending System.String
    public static bool IsValidEmail(this string email)
    {
        if (string.IsNullOrWhiteSpace(email)) return false;
        return email.Contains("@") && email.Contains(".");
    }
}

// Usage:
string userEmail = "dev@example.com";
bool isValid = userEmail.IsValidEmail(); // Called like an instance method!
```

Q15. Explain JWT Authentication. Where is the token stored?

JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact, URL-safe container for securely transmitting claims between two parties. It consists of 3 parts separated by dots: `Header.Payload.Signature`.

- **Header:** Specifies signing algorithm (e.g., HS256, RS256) and token type.
- **Payload:** Contains claims (User ID, Roles, Expiration time `exp`).
- **Signature:** Created by hashing Header + Payload with a secret key to prevent tampering.

> **Where is the JWT Token stored on the Client?**
>
> - **HttpOnly, Secure Cookie (Recommended):** Prevents Cross-Site Scripting (XSS) attacks because JavaScript cannot read HttpOnly cookies. Requires SameSite settings to mitigate CSRF.
> - **Browser Storage (LocalStorage / SessionStorage):** Easy to implement, but vulnerable to XSS attacks if the app has script injection vulnerabilities.

Q16. Why is DbContext registered as Scoped?

In Entity Framework Core, `DbContext` is registered with a **Scoped lifetime** in ASP.NET Core for critical technical reasons:

1. **Unit of Work & Change Tracker:** `DbContext` maintains an in-memory change tracker. A scoped lifetime ensures that all operations during a single HTTP request share the same tracking context and commit together via `SaveChangesAsync()`.
2. **Thread Safety:** `DbContext` is *not thread-safe*. Registering as Singleton would cause race conditions and corrupted data when multiple requests run concurrently.
3. **Memory Management:** Registering as Transient would create multiple DbContext instances per request, losing change tracking across services and wasting database connection pool connections.

Q17. Explain your Project Architecture.

A modern production-grade .NET application typically follows **Clean Architecture (Onion / Hexagonal Architecture)** to separate concerns and enforce the Dependency Inversion Principle.

- **Domain Layer (Core):** Central layer containing Entities, Value Objects, Domain Events, and Enums. Has zero external dependencies.
- **Application Layer:** Contains Use Cases, DTOs, CQRS Handlers (MediatR), Interfaces (IRepository, ICurrentUserService), and FluentValidation.
- **Infrastructure Layer:** Contains external concerns: Entity Framework Core DbContext, Migrations, Redis Caching, Email Services, and Payment Gateways.
- **Presentation / API Layer:** Controllers, Minimal APIs, Middleware, Program.cs, and Swagger.

Q18. What is Code First? Can we create database tables without SQL scripts?

Code First is an Entity Framework Core development approach where developers define domain model C# classes first. EF Core uses these C# classes to generate database schemas automatically using *Migrations*.

Can we create database tables without SQL scripts?
Yes! By running EF Core Migration commands (`dotnet ef database update` or `context.Database.MigrateAsync()` at app startup), EF Core dynamically generates DDL scripts under the hood and executes them against SQL Server without requiring manual SQL creation scripts.

```
// C# Model
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
}

// Terminal Command to create DB tables automatically:
// dotnet ef migrations add InitialCreate
// dotnet ef database update
```

Q19. Difference between PUT and PATCH?

Feature	PUT	PATCH
Purpose	Replaces the **entire resource** payload.	Applies **partial modifications** to a resource.
Payload Requirement	Must send all resource fields (missing fields may be set to null/default).	Sends only the specific fields that need updating.
Idempotency	**Idempotent:** Sending identical PUT requests multiple times yields the same result.	**Non-Idempotent (by spec):** Though often implemented idempotently, operations like appending to a list are non-idempotent.

Q20. Why do we use POST instead of GET?

Aspect	GET	POST
Purpose	Used purely for **retrieving** data from server.	Used for **submitting / creating / processing** data.
Data Location	Data sent in URL query string (e.g., `?id=10&name=john`).	Data sent in request HTTP Body.
Security	Sensitive data in URLs is logged in web server logs, browser history, and referrers.	Request body is encrypted over HTTPS and not logged in URL histories.
Data Limit	Restricted by URL length limit (~2048 characters).	No fixed length limit (suitable for file uploads & heavy JSON).
Caching & Idempotency	Idempotent and cacheable by browsers and CDNs.	Non-idempotent and not cached by default.

Q21. Explain the ASP.NET Core Request Life Cycle.

When an HTTP request hits an ASP.NET Core web application, it moves through the following pipeline:

1. **Kestrel Web Server:** Receives raw HTTP bytes from network and constructs HttpContext.
2. **Middleware Pipeline:** Request flows sequentially through registered middleware (Logging → ExceptionHandler → HSTS → HTTPS → StaticFiles → Routing → CORS → Authentication → Authorization).
3. **Endpoint Selection & Routing:** Matching route selects target Controller Action.
4. **Filter Pipeline Execution:**
   - Authorization Filters execute.
   - Resource Filters execute.
   - Model Binding & Validation occurs.
   - Action Filters execute (`OnActionExecuting`).
   - Controller Action Method executes.
   - Action Filters execute (`OnActionExecuted`).
   - Result Filters & Exception Filters process output.
5. **Response Generation:** Response travels back backwards through middleware pipeline to Kestrel and back to client.

Q22. Explain LINQ and its methods.

LINQ (Language Integrated Query) is a C# feature providing uniform query syntax to filter, transform, and aggregate data across collections, SQL databases, XML, etc.

Common LINQ Methods:

- `Where()` : Filters elements based on a predicate.
- `Select()` : Projects/transforms elements into new shapes (DTOs).
- `FirstOrDefault()` : Returns first element matching condition or default (null/0).
- `GroupBy()` : Groups elements by a specified key.
- `OrderBy() / OrderByDescending()` : Sorts elements.
- `Any() / All()` : Evaluates if any or all elements satisfy a condition.
- `SelectMany()` : Flattens nested collections into a single collection.

```
var topStudents = students
    .Where(s ⇒ s.Grade ≥ 85)
    .OrderByDescending(s ⇒ s.Grade)
    .Select(s ⇒ new { s.Name, s.Grade })
    .Take(5)
    .ToList();
```

Q23. Difference between Eager, Lazy and Explicit Loading?

In EF Core, loading related data from database relationships can be achieved in 3 ways:

Loading Type	Mechanism	Execution & Example
Eager Loading	Related data is loaded from the database as part of the initial SQL query using `Include()` / `ThenInclude()` . (SQL JOIN).	`_context.Orders.Include(o ⇒ o.OrderItems).ToList();`
Lazy Loading	Related data is transparently loaded on-demand from database when navigation property is accessed. (Requires `UseLazyLoadingProxies()` & `virtual` properties).	`var items = order.OrderItems; // Triggers automatic SQL query!`
Explicit Loading	Related data is explicitly fetched from database at a later time using `Entry().Collection().LoadAsync()` .	`await _context.Entry(order).Collection(o ⇒ o.Items).LoadAsync();`

Q24. Difference between ViewData, ViewBag and TempData?

Feature	ViewData	ViewBag	TempData
Type	Dictionary of objects (`ViewDataDictionary`). Requires type casting.	Dynamic property wrapper around ViewData. Uses C# 4 `dynamic` .	Dictionary based on `ITempDataProvider` (usually session/cookie).
Life Span	Current Request only (Controller to View).	Current Request only (Controller to View).	Persists across **2 consecutive requests** (useful for HTTP Redirections).
Type Safety	No compile-time check (requires casting).	No compile-time check (evaluated at runtime).	No compile-time check (requires casting).

Q25. Difference between Session and Cookies?

Feature	Cookie	Session
Storage Location	Stored directly on the **Client Browser**.	Stored on the **Web Server** (RAM, Redis, SQL). Session ID stored in client cookie.
Data Size Limit	Limited to ~4KB per cookie.	No fixed limit (bounded only by server storage capacity).
Security	Vulnerable to tampering unless encrypted / HttpOnly.	High security; client only holds non-sensitive session ID key.
Expiration	Can persist for months/years based on set expiration date.	Expires when browser closes or idle timeout occurs (default ~20 mins).

Q26. Difference between readonly, const and static?

Keyword	Evaluation Time	Initialization	Memory Allocation
`const`	Compile-Time constant.	Must be initialized at variable declaration.	Value is directly embedded into calling IL code. No memory allocation.
`readonly`	Run-Time constant.	Can be initialized at declaration OR inside Class Constructor.	Allocated instance memory. Can hold runtime object values.
`static`	Run-Time scope modifier.	Initialized when class is first loaded into memory.	Shared across all instances of the class. Single memory location.

Q27. Difference between SOAP and REST API?

Feature	SOAP API	REST API
Protocol vs Architecture	Strict Protocol (Simple Object Access Protocol).	Flexible Architectural Style (Representational State Transfer).
Data Format	Strictly XML formatted payload defined by WSDL contract.	Supports JSON, XML, Plain Text, HTML (JSON is standard).
Transport Protocols	HTTP, SMTP, TCP, UDP.	Exclusively HTTP / HTTPS.
Security	Built-in WS-Security standards (Enterprise grade).	Relies on HTTPS, OAuth2, JWT, and API Keys.
Performance	Slower due to heavy XML serialization overhead.	Fast and lightweight JSON transfer.

Q28. What is CORS?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism (Same-Origin Policy enforcement) that controls whether web apps running on one origin (domain, protocol, port e.g. `http://localhost:3000`) can request resources from a different origin (e.g. `https://api.myapp.com`).

How to configure CORS in ASP.NET Core:

```
// Program.cs
builder.Services.AddCors(options ⇒
{
    options.AddPolicy("AllowFrontendApp", policy ⇒
    {
        policy.WithOrigins("https://myapp.com", "http://localhost:3000")
              .AllowAnyHeader()
              .AllowAnyMethod()
              .AllowCredentials();
    });
});

var app = builder.Build();
app.UseCors("AllowFrontendApp"); // Place between UseRouting and UseAuthorization
```

Q29. Difference between String and StringBuilder?

Feature	System.String	System.Text.StringBuilder
Mutability	**Immutable** (Cannot be changed after creation).	**Mutable** (Modifies buffer in-place).
Modification Cost	Every modification creates a brand new string object on Heap, triggering Garbage Collection.	Appends/modifies character buffer dynamically without allocating new objects.
Performance	Fast for small static strings or concatenations (<= 3 operations).	Extremely high performance for heavy string manipulation or loop concatenations.

```
// Bad: Allocates 10,000 objects on heap!
string text = "";
for(int i=0; i<10000; i++) text += i.ToString();

// Good: Single memory buffer modified in place!
var sb = new StringBuilder();
for(int i=0; i<10000; i++) sb.Append(i);
string result = sb.ToString();
```

Q30. How does CLR work?

The **CLR (Common Language Runtime)** is the execution engine of the .NET framework that manages program execution. Execution steps:

1. **Source Code Compilation:** C# code is compiled by Roslyn compiler into *CIL / IL (Common Intermediate Language)* and stored in assemblies (.dll/.exe) alongside Metadata.
2. **JIT Compilation:** When the app runs, the CLR's **JIT (Just-In-Time) Compiler** converts IL into native CPU machine code dynamically.
3. **Managed Services Provided by CLR:**
   - **Garbage Collection (GC):** Automatic memory management & object deallocation.
   - **Type Safety Enforcement:** Ensures objects access valid memory boundaries.
   - **Exception Handling:** Structured try-catch handling across languages.
   - **Thread & Security Management:** Manages thread pools and access rights.

Q31. Explain SOLID Principles.

SOLID represents 5 object-oriented design principles that promote maintainable, flexible, and scalable software architecture:

- **S - Single Responsibility Principle (SRP):** A class should have one, and only one, reason to change.
- **O - Open/Closed Principle (OCP):** Software entities should be open for extension, but closed for modification.
- **L - Liskov Substitution Principle (LSP):** Subtypes must be substitutable for their base types without breaking application behavior.
- **I - Interface Segregation Principle (ISP):** Clients should not be forced to depend on interfaces they do not use (prefer small, role-specific interfaces).
- **D - Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules; both should depend on abstractions (interfaces).

Q32. What is API Throttling?

API Throttling is a rate-limiting policy used to control the rate of requests a client can make to an API within a specified timeframe (e.g., maximum 100 requests per minute).

> **Why implement Throttling?**
>
> - **Prevent Denial of Service (DoS):** Protects backend servers from being overwhelmed by malicious or buggy bots.
> - **Fair Usage:** Ensures equal bandwidth sharing among API consumers.
> - **Monetization:** Enforces API tier usage limits (e.g., Basic vs Premium tier).

When limit is exceeded, server returns `HTTP 429 Too Many Requests` with a `Retry-After` header.

Section 2: ASP.NET Core Advanced, Microservices & Architecture

Q33. Authentication vs Authorization?

Aspect	Authentication	Authorization
Definition	Verifies **WHO** the user is (Identity Verification).	Determines **WHAT** permissions the verified user has.
Process	Validates credentials (Username/Password, OTP, JWT, OAuth).	Checks user roles, claims, or policies (e.g. Admin, Manager).
Failure Result	Returns `HTTP 401 Unauthorized`.	Returns `HTTP 403 Forbidden`.

Q34. JWT vs OAuth?

Feature	JWT (JSON Web Token)	OAuth 2.0
Type	**Token Format / Standard** for representing claims.	**Authorization Framework / Protocol** for granting third-party access.
Role	Carries user identity payload securely between client and server.	Delegates authorization without sharing password (e.g., "Sign in with Google").
Relationship	JWT is frequently used *as the token format* inside OAuth 2.0 implementations.	OAuth 2.0 defines the workflow/handshake that issues JWT tokens.

Q35. Access Token vs Refresh Token?

Feature	Access Token	Refresh Token
Purpose	Used by client to authorize requests to protected API endpoints.	Used to obtain a new Access Token without prompting user to re-login.
Lifespan	Short-lived (e.g., 15 - 30 minutes) for security.	Long-lived (e.g., 7 - 30 days).
Storage & Safety	Sent in HTTP Authorization header (`Bearer <token>`).	Stored securely in HttpOnly, SameSite cookie or encrypted DB table.

Q36. What is CQRS?

CQRS (Command Query Responsibility Segregation) is an architectural pattern that separates read operations (Queries) from write/update operations (Commands).

- **Commands:** Change the state of the application (Create/Update/Delete). Do not return domain data.
- **Queries:** Retrieve data from database (Read-only). Never alter application state.

> **Benefits of CQRS:**
> - Optimized Data Models: Use normalized SQL DB for writes and denormalized Redis/Elasticsearch for fast reads.
> - Independent Scaling: Read and write workloads can scale independently.

Q37. What is MediatR?

MediatR is a popular open-source .NET library implementing the *Mediator Pattern*. It enables completely decoupled in-process messaging between callers and handlers.

```
// 1. Command Request
public record CreateUserCommand(string Name, string Email) : IRequest<int>;

// 2. Command Handler
public class CreateUserHandler : IRequestHandler<CreateUserCommand, int>
{
    private readonly AppDbContext _db;
    public CreateUserHandler(AppDbContext db) => _db = db;

    public async Task<int> Handle(CreateUserCommand request, CancellationToken
cancellationToken)
    {
        var user = new User { Name = request.Name, Email = request.Email };
        _db.Users.Add(user);
        await _db.SaveChangesAsync(cancellationToken);
        return user.Id;
    }
}

// 3. Controller Dispatcher
[HttpPost]
public async Task<IActionResult> Create(CreateUserCommand command)
{
    var id = await _mediator.Send(command); // Controller has zero direct service
dependencies!
    return Ok(id);
}
```

Q38. What is Clean Architecture?

Clean Architecture is a software design pattern proposed by Robert C. Martin (Uncle Bob). It organizes code into concentric rings where dependencies only point **inward** towards domain models.

- **Domain (Core):** Central business rules and entities. No dependencies.
- **Application:** Business logic, interfaces, CQRS, and use cases. Depends only on Domain.
- **Infrastructure:** Persistence (EF Core), Caching, External APIs. Implements Application interfaces.
- **Presentation (UI/API):** Controllers, Web API, UI views. Entry point.

Q39. What is Repository Pattern?

The **Repository Pattern** mediates between domain and data mapping layers, acting as an in-memory collection of domain objects.

```
public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T?> GetByIdAsync(int id);
    Task AddAsync(T entity);
    void Remove(T entity);
}

public class Repository<T> : IRepository<T> where T : class
{
    protected readonly AppDbContext _context;
    public Repository(AppDbContext context) => _context = context;

    public async Task<T?> GetByIdAsync(int id) => await _context.Set<T>
().FindAsync(id);
    public async Task AddAsync(T entity) => await _context.Set<T>().AddAsync(entity);
}
```

Q40. What is Unit of Work?

The **Unit of Work Pattern** maintains a list of database operations within a business transaction and commits all changes together in a single atomic database operation (e.g. `SaveChangesAsync()`).

```
public interface IUnitOfWork : IDisposable
{
    IUserRepository Users { get; }
    IOrderRepository Orders { get; }
    Task<int> CompleteAsync(); // Single transaction commit!
}
```

Q41. What is AutoMapper?

AutoMapper is an object-to-object mapping library for .NET used to automatically copy values from one object type to another (e.g., mapping EF Core Domain Entities to Response DTOs).

```
// Mapping Profile Setup
public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<User, UserDto>()
            .ForMember(dest ⇒ dest.FullName, opt ⇒ opt.MapFrom(src ⇒ $"
{src.FirstName} {src.LastName}"));
    }
}

// Usage in controller:
UserDto dto = _mapper.Map<UserDto>(userEntity);
```

Q42. What is Redis?

Redis (Remote Dictionary Server) is an ultra-fast, open-source, in-memory key-value data store used as a database, distributed cache, and message broker. Data is held directly in RAM yielding sub-millisecond read/write speeds.

> **Key Redis Use Cases:**
> - Distributed Caching across load-balanced microservices.
> - Session state persistence.
> - API Rate limiting counters.
> - Pub/Sub real-time messaging.

Q43. What is SignalR?

ASP.NET Core SignalR is an open-source library that adds real-time web functionality to applications. It allows server-side code to push asynchronous content updates to connected clients instantly.

- **Transports:** Uses *WebSockets* primarily, automatically falling back to Server-Sent Events or Long Polling if WebSockets aren't supported.
- **Use Cases:** Live chat applications, real-time financial dashboards, multiplayer games, collaborative editing, and push notification alerts.

Q44. What is Serilog?

Serilog is a structured logging framework for .NET applications. Unlike traditional text-file logging, Serilog records logs as structured JSON objects containing name-value properties.

```
// Structured log statement
_logger.LogInformation("Order {OrderId} processed for Customer {CustomerId} with total
{Amount}",
    order.Id, customer.Id, order.TotalAmount);

// Output sent to sinks (Seq, Elasticsearch, Datadog):
// { "OrderId": 1042, "CustomerId": 99, "Amount": 150.00, "Timestamp": "2026-08-
06T18:52:00Z" }
```

Q45. What are Health Checks?

Health Checks in ASP.NET Core provide HTTP endpoints (e.g. `/health`) that expose the health status of an application and its downstream dependencies (SQL database, Redis cache, external payment gateway).

```
// Program.cs
builder.Services.AddHealthChecks()
    .AddSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
    .AddRedis("localhost:6379");

var app = builder.Build();
app.MapHealthChecks("/health"); // Returns 200 Healthy or 530 Unhealthy
```

Q46. What is Docker?

Docker is an open-source containerization platform that packages an application and all its dependencies (runtime, libraries, configs) into a single lightweight *Docker Container*. This guarantees that code runs identically across dev, staging, and production environments.

Q47. What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform designed to automate deployment, scaling, load balancing, health monitoring, and management of containerized applications (Docker) across cluster nodes.

Q48. RabbitMQ vs Kafka?

Feature	RabbitMQ	Apache Kafka
Architecture	Traditional **Smart Broker / Dumb Consumer** message queue.	Distributed **Log Stream Broker / Smart Consumer** event stream.
Message Consumption	Messages are deleted from queue once acknowledged by worker.	Messages persist in log topic based on retention policy (can replay).
Throughput	High (~10k-50k msgs/sec). Complex routing capabilities (AMQP).	Ultra-High (100k - 1M+ msgs/sec). Stream processing architecture.
Use Case	Transactional task queues, microservice RPC messaging.	Real-time event streaming, log processing, big data pipelines.

Q49. What is gRPC?

gRPC (Google Remote Procedure Call) is a high-performance, open-source RPC framework. It uses **HTTP/2** for transport, **Protocol Buffers (Protobuf)** as binary serialization format, and supports contract-first API development.

> **Why gRPC over REST?**
> - Up to 7x-10x faster than REST/JSON due to compact binary Protobuf payload.
> - Built-in support for bi-directional streaming over single HTTP/2 connection.
> - Ideal for high-throughput internal microservice-to-microservice communication.

Q50. What is API Versioning?

API Versioning allows developers to release breaking changes to API endpoints without disrupting existing clients relying on previous API contracts.

Versioning Strategies:

- **URI Versioning:** `api/v1/products` vs `api/v2/products` (Most popular).
- **Header Versioning:** `X-Api-Version: 2.0`.
- **Query String Versioning:** `api/products?api-version=2.0`.

Q51. What is Swagger/OpenAPI?

Swagger (OpenAPI Specification) provides a machine-readable JSON/YAML description of API endpoints, HTTP methods, parameters, request/response models, and auth schemes, paired with an interactive UI allowing developers to test API endpoints in browser.

Q52. What is Idempotency?

An HTTP operation is **Idempotent** if making multiple identical requests has the exact same side-effect on the server as making a single request.

- **GET, PUT, DELETE, HEAD, OPTIONS:** Idempotent.
- **POST:** Non-idempotent (e.g. sending 3 POST requests creates 3 new records or charges credit card 3 times).

Q53. What is CancellationToken?

A `CancellationToken` enables cooperative cancellation of long-running asynchronous operations. In ASP.NET Core, if a web user cancels a request or closes the browser tab, the `HttpContext.RequestAborted` token signals EF Core to abort SQL queries immediately, saving CPU and DB resources.

```
[HttpGet("data")]
public async Task<IActionResult> GetData(CancellationToken cancellationToken)
{
    var data = await _dbContext.LargeTable
        .ToListAsync(cancellationToken); // Aborts SQL query if client disconnects!
    return Ok(data);
}
```

Q54. What are Minimal APIs?

Minimal APIs (introduced in .NET 6) allow developers to build HTTP APIs with minimal code and overhead by defining endpoints using lambdas directly in `Program.cs` without controller boilerplate.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/users/{id}", async (int id, AppDbContext db) =>
    await db.Users.FindAsync(id) is User user ? Results.Ok(user) :
Results.NotFound());

app.Run();
```

Q55. What are Background Services?

A **Background Service** in ASP.NET Core is a long-running task managed by the host. It implements `IHostedService` or derives from `BackgroundService` to execute tasks asynchronously in the background (e.g., periodic cleanup jobs, message queue processing).

```
public class EmailQueueProcessor : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            // Process queued emails every 30 seconds
            await Task.Delay(30000, stoppingToken);
        }
    }
}
```

Q56. Microservices vs Monolithic Architecture?

Feature	Monolithic Architecture	Microservices Architecture
Deployment	Single unified codebase deployed as one executable unit.	Independently deployable services communicating over network.
Database	Single shared central database.	Database per service pattern.
Scalability	Must scale the whole application monolith.	Scale individual high-demand services independently.
Complexity	Low initial development and deployment complexity.	High operational and network/distributed tracing complexity.

Q57. Memory Cache vs Distributed Cache?

Feature	In-Memory Cache (IMemoryCache)	Distributed Cache (IDistributedCache)
Storage Location	Stored inside web app process RAM.	Stored in dedicated external server cluster (e.g., Redis / NCache).
Scope	Local to a single server instance.	Shared across multiple load-balanced web app servers.
Data Persistence	Cache lost when app pool restarts.	Cache survives app pool deployments/restarts.

Q58. What is Response Caching?

Response Caching sets HTTP headers (such as `Cache-Control`, `Expires`, and `Vary`) to instruct client browsers, proxies, and CDN networks to cache HTTP responses locally for a specified period.

Q59. What is Output Caching?

Output Caching (introduced in .NET 7) is a powerful server-side caching mechanism that caches complete HTTP response payloads on the ASP.NET Core server directly, avoiding re-execution of endpoint routing, controller code, and SQL queries.

Q60. What is Polly Retry Policy?

Polly is a .NET resilience and transient-fault-handling library. A **Retry Policy** allows developers to configure automatic retries with exponential backoff when calls to external services fail transiently.

```
// Policy definition
var retryPolicy = HttpPolicyExtensions
    .HandleTransientHttpError()
    .WaitAndRetryAsync(3, retryAttempt ⇒ TimeSpan.FromSeconds(Math.Pow(2,
retryAttempt)));

// Registration with HttpClient
builder.Services.AddHttpClient<IPaymentService, PaymentService>()
    .AddPolicyHandler(retryPolicy);
```

Q61. Explain Dependency Injection Lifetimes.

Lifetime	Creation & Behavior
Transient (`AddTransient`)	Created **every single time** they are requested. Ideal for lightweight stateless services.
Scoped (`AddScoped`)	Created **once per HTTP Request** scope. Shared across classes in that request. Ideal for DbContext and repositories.
Singleton (`AddSingleton`)	Created **once on application startup** and shared across all subsequent HTTP requests until app stops. Ideal for in-memory caches or config objects.

Q62. Explain API Rate Limiting.

Native Rate Limiting in .NET 7+ provides middleware algorithms to throttle requests:

- **Fixed Window:** Allows max N requests in fixed time frame (e.g., 100 reqs/min).
- **Sliding Window:** Divides window into segments for smooth limiting without burst boundary spikes.
- **Token Bucket:** Adds tokens at regular intervals; requests consume tokens. Supports bursts up to bucket capacity.
- **Concurrency Limiter:** Limits maximum simultaneous active requests.

Section 3: SQL Server Database Engineering & Optimization

Q63. How do you optimize a Stored Procedure?

Key strategies to optimize SQL Server Stored Procedures:

- **Avoid `SELECT *`:** Specify required column names explicitly to reduce I/O and network payload.
- **Use Appropriate Indexes:** Create non-clustered indexes on columns used in `WHERE`, `JOIN`, and `ORDER BY` clauses.
- **Prevent Parameter Sniffing:** Use `OPTION (RECOMPILE)` or declare local variables inside procedure.
- **Use `SET NOCOUNT ON`:** Prevents sending done-in-proc messages for every statement.
- **Avoid Functions in WHERE Clauses:** Functions like `WHERE YEAR(OrderDate) = 2026` make queries non-sargable (cannot use index). Use `WHERE OrderDate ≥ '2026-01-01'` instead.
- **Use CTEs / Temp Tables wisely:** Replace cursors with set-based operations.

Q64. How do you measure Stored Procedure execution time?

```
-- Method 1: Using SET STATISTICS TIME & IO
SET STATISTICS TIME ON;
SET STATISTICS IO ON;

EXEC sp_GetCustomerOrders @CustomerId = 105;

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;

-- Method 2: Dynamic Management Views (DMVs) for overall execution stats
SELECT
    OBJECT_NAME(st.objectid, st.dbid) AS ProcedureName,
    qs.execution_count,
    qs.total_elapsed_time / 1000.0 AS TotalTimeMs,
    (qs.total_elapsed_time / qs.execution_count) / 1000.0 AS AvgTimeMs
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE st.objectid = OBJECT_ID('sp_GetCustomerOrders');
```

Q65. Clustered vs Non-Clustered Index?

Feature	Clustered Index	Non-Clustered Index
Physical Storage	Physically sorts and stores actual data rows of table in index order.	Stored separately from physical data table; contains key values and pointers to data rows.
Limit per Table	Max **1** per table (usually Primary Key).	Max **999** per table.
Leaf Nodes	Leaf nodes contain the actual data rows.	Leaf nodes contain index key values + row locators (RID or Clustered Index Key).
Lookup Speed	Faster for range scans because data is stored sequentially.	Requires bookmark lookup (Key Lookup) if query selects unindexed columns.

Q66. How do Transactions work?

A **Transaction** in SQL Server is a sequence of operations executed as a single logical unit of work adhering to ACID properties. If any statement fails, the entire transaction is rolled back.

```
BEGIN TRANSACTION;

BEGIN TRY
    UPDATE Accounts SET Balance = Balance - 500 WHERE AccountId = 1;
    UPDATE Accounts SET Balance = Balance + 500 WHERE AccountId = 2;

    COMMIT TRANSACTION; -- Saves changes permanently
    PRINT 'Transfer Successful!';
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION; -- Undoes all changes on error
    PRINT 'Transfer Failed: ' + ERROR_MESSAGE();
END CATCH;
```

Q67. Function vs Stored Procedure?

Feature	User Defined Function (UDF)	Stored Procedure (SP)
Return Value	Must return a value or table.	Optional return value (or zero/multiple result sets).
SQL Execution Scope	Can be called within `SELECT`, `WHERE`, and `JOIN` statements.	Executed standalone using `EXEC` or `EXECUTE`.
DML Operations	Cannot perform INSERT, UPDATE, DELETE on physical database tables.	Can perform full DML/DDL modifications and transaction control.
Transactions	Cannot manage transactions (no `BEGIN TRAN`).	Supports transaction management (`BEGIN/COMMIT/ROLLBACK`).

Q68. Stored Procedure vs View?

Feature	View	Stored Procedure
Definition	Virtual table based on a saved `SELECT` query.	Precompiled batch of executable T-SQL statements.
Parameters	Does not accept input parameters.	Accepts input and output parameters.
Logic Complexity	Used for query simplification and column security.	Can hold complex procedural logic, loops, temp tables, and transactions.

Q69. Find the 5th Highest Salary.

```
-- Method 1: Using DENSE_RANK() Window Function (Recommended)
WITH RankedSalaries AS (
    SELECT
        EmployeeId, FirstName, Salary,
        DENSE_RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
    FROM Employees
)
SELECT EmployeeId, FirstName, Salary
FROM RankedSalaries
WHERE SalaryRank = 5;

-- Method 2: Using OFFSET / FETCH (SQL Server 2012+)
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
OFFSET 4 ROWS FETCH NEXT 1 ROW ONLY;
```

Q70. Find the Top 3 Highest Salaries.

```
-- Method 1: Using DENSE_RANK() to include ties
WITH RankedSalaries AS (
    SELECT
        EmployeeId, FirstName, Salary,
        DENSE_RANK() OVER (ORDER BY Salary DESC) AS SalaryRank
    FROM Employees
)
SELECT EmployeeId, FirstName, Salary
FROM RankedSalaries
WHERE SalaryRank ⩽ 3;

-- Method 2: Simple TOP clause
SELECT TOP 3 WITH TIES EmployeeId, FirstName, Salary
FROM Employees
ORDER BY Salary DESC;
```

Q71. What are SQL Constraints?

SQL Constraints are rules enforced on table columns to maintain data accuracy, reliability, and integrity.

- `NOT NULL` : Ensures column cannot hold NULL values.
- `UNIQUE` : Guarantees all values in column are distinct.
- `PRIMARY KEY` : Uniquely identifies each record (NOT NULL + UNIQUE).
- `FOREIGN KEY` : Enforces referential integrity between tables.
- `CHECK` : Ensures values satisfy a logical expression (e.g. `CHECK (Age ⩾ 18)`).
- `DEFAULT` : Provides default value when none is specified.

Q72. DELETE vs TRUNCATE vs DROP?

Feature	DELETE	TRUNCATE	DROP
Command Type	DML (Data Manipulation Language).	DDL (Data Definition Language).	DDL (Data Definition Language).
Operation	Removes specified rows one by one.	Removes ALL rows by deallocating data pages.	Removes entire table structure & data from database.
WHERE Clause	Supported (can delete specific rows).	Not supported (clears entire table).	Not applicable.
Identity Reset	Does NOT reset IDENTITY seed.	Resets IDENTITY seed back to initial value.	Removes identity and entire table object.
Speed & Logging	Slower; logs each row deletion individually in transaction log.	Extremely fast; logs page deallocations only.	Instant.

Q73. Why can DELETE be rolled back but TRUNCATE usually cannot?

Myth Busted: Both `DELETE` and `TRUNCATE` **CAN be rolled back** if executed inside an active explicit transaction (`BEGIN TRANSACTION`)!

> **Key Logging Difference:**
> `DELETE` records every deleted row in the Transaction Log, making it slower but fully recoverable. `TRUNCATE` is a DDL operation that logs page deallocations rather than individual rows. If committed, TRUNCATE transaction log space can be recycled immediately, but while inside an open transaction block, `ROLLBACK` restores the deallocated pages!

Q74. What is CTE?

A **CTE (Common Table Expression)** is a temporary result set defined using the `WITH` clause that can be referenced within a `SELECT` , `INSERT` , `UPDATE` , or `DELETE` statement.

```
WITH HighValueOrders AS (
    SELECT CustomerId, SUM(TotalAmount) AS TotalSpent
    FROM Orders
    GROUP BY CustomerId
    HAVING SUM(TotalAmount) > 10000
)
SELECT c.Name, h.TotalSpent
FROM HighValueOrders h
JOIN Customers c ON c.Id = h.CustomerId;
```

Q75. Temp Table vs Table Variable?

Feature	Temp Table (`#TempTable`)	Table Variable (`@TableVar`)
Storage Location	Stored in `tempdb` system database.	Stored in `tempdb` (memory with spillover to tempdb).
Scope	Session scope or procedure scope.	Local variable scope (current batch/procedure execution).
Indexes & Stats	Supports Clustered & Non-Clustered Indexes and Statistics.	Supports Primary Key/Unique constraints (no statistics).
Best Used For	Large datasets (> 100 rows) requiring complex joins & indexing.	Small lookup datasets (< 100 rows).

Q76. Find Duplicate Records using CTE.

```
-- Query to identify and delete duplicate rows based on Email column
WITH DuplicateCTE AS (
    SELECT
        Id, Email,
        ROW_NUMBER() OVER (PARTITION BY Email ORDER BY Id) AS RowNum
    FROM Users
)
-- RowNum > 1 represents duplicate entries
SELECT * FROM DuplicateCTE WHERE RowNum > 1;

-- To DELETE duplicates:
-- DELETE FROM DuplicateCTE WHERE RowNum > 1;
```

Q77. ROW_NUMBER vs RANK vs DENSE_RANK?

Given values: `100, 100, 90, 80`

Function	Behavior	Output Sequence
`ROW_NUMBER()`	Assigns unique sequential integer to every row regardless of ties.	`1, 2, 3, 4`
`RANK()`	Assigns same rank to ties, but **skips rank numbers** after ties.	`1, 1, 3, 4`
`DENSE_RANK()`	Assigns same rank to ties, **without skipping** subsequent ranks.	`1, 1, 2, 3`

Q78. EXISTS vs IN?

Feature	EXISTS	IN
Evaluation Method	Short-circuits immediately upon finding the first matching row (returns TRUE/FALSE).	Evaluates subquery fully and checks if target value is in returned list.
Handling NULLs	Handles `NULL` values cleanly in subquery.	If subquery returns `NULL`, `NOT IN` returns empty set!
Performance	Significantly faster for large subqueries.	Fast for small, hardcoded literal lists.

Q79. INNER JOIN vs LEFT JOIN vs RIGHT JOIN?

- **INNER JOIN:** Returns only matching records existing in BOTH left and right tables.
- **LEFT JOIN (LEFT OUTER JOIN):** Returns ALL records from left table, plus matched records from right table (NULLs for unmatched right columns).
- **RIGHT JOIN (RIGHT OUTER JOIN):** Returns ALL records from right table, plus matched records from left table (NULLs for unmatched left columns).

Q80. UNION vs UNION ALL?

Feature	UNION	UNION ALL
Duplicates	Combines result sets and **removes duplicate rows**.	Combines result sets and **retains duplicate rows**.
Sorting	Performs implicit Distinct Sort operation in tempdb.	No sorting or distinct checking.
Performance	Slower due to duplicate removal overhead.	Much faster.

Q81. What is Normalization?

Normalization is the process of organizing database tables to minimize data redundancy and improve data integrity.

- **1NF (First Normal Form):** Atomic values per column; no repeating groups.
- **2NF (Second Normal Form):** Must be in 1NF + all non-key columns depend fully on Primary Key (eliminates partial dependency).
- **3NF (Third Normal Form):** Must be in 2NF + eliminate transitive dependencies (non-key columns depend only on PK).

Q82. What is Denormalization?

Denormalization is the deliberate optimization technique of adding redundant data or grouping tables in a normalized database to reduce expensive `JOIN` operations and speed up read query performance (common in Data Warehousing and OLAP systems).

Q83. What are ACID Properties?

- **Atomicity:** All statements in a transaction succeed together, or all fail and rollback (All-or-Nothing).
- **Consistency:** Database transitions from one valid state to another, maintaining constraints and invariants.
- **Isolation:** Concurrent transactions execute independently without interfering with each other (Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable).
- **Durability:** Once committed, transaction changes survive system crashes and power outages.

Q84. What is Deadlock?

A **Deadlock** occurs when two or more concurrent transactions hold locks on resources that the other transaction requires, creating a cyclic dependency where neither can proceed.

> **How SQL Server Handles Deadlocks:**
> SQL Server's Deadlock Monitor automatically detects cyclic lock waits, selects a *deadlock victim* (transaction with lowest rollback cost), terminates it, and throws Error 1205 to let the remaining transaction proceed.

Q85. How do you read an Execution Plan?

Execution Plans illustrate the physical operations SQL Server query optimizer selects to execute a query:

- Read execution plans from **Right to Left** and **Top to Bottom**.
- Look out for high cost operators: **Table Scan** (missing index), **Key Lookup** (non-clustered index missing columns), **Implicit Conversion** (data type mismatch).

Q86. What is Index Fragmentation?

Index Fragmentation occurs when index pages become out of logical order due to frequent INSERT, UPDATE, or DELETE operations.

- **Fragmentation 5% to 30%:** Reorganize Index (`ALTER INDEX REORGANIZE` - Online operation).
- **Fragmentation > 30%:** Rebuild Index (`ALTER INDEX REBUILD` - Creates new index structure).

Q87. What are Window Functions?

Window Functions perform calculations across a set of table rows related to the current row without collapsing rows into a single summary output (unlike `GROUP BY`).

```
SELECT
    EmployeeId, DepartmentId, Salary,
    AVG(Salary) OVER (PARTITION BY DepartmentId) AS AvgDeptSalary,
    SUM(Salary) OVER (PARTITION BY DepartmentId ORDER BY Salary) AS RunningTotal
FROM Employees;
```

Q88. What is a Cursor?

A **Cursor** is a T-SQL database object used to traverse and process query result rows one row at a time in a loop.

> **Why avoid Cursors?**
> Cursors are extremely slow because they process row-by-row instead of taking advantage of SQL Server's set-based relational engine. Replace cursors with set-based SQL queries, CTEs, or Window Functions.

Q89. How do you improve SQL Query Performance?

1. Create proper Indexes (Clustered & Non-clustered with `INCLUDE` columns).
2. Avoid `SELECT *` and wildcard leading LIKE queries (e.g. `LIKE '%abc'`).
3. Use Set-based operations instead of loops/cursors.
4. Ensure queries are Sargable (avoid wrapping columns in functions in WHERE clause).
5. Keep table statistics updated (`UPDATE STATISTICS`).

Q90. Primary Key vs Unique Key?

Feature	Primary Key	Unique Key
NULL Values	Cannot accept `NULL` values.	Accepts max **1 NULL** value (in SQL Server).
Default Index	Creates a **Clustered Index** by default.	Creates a **Non-Clustered Index** by default.
Limit per Table	Max **1** Primary Key per table.	Multiple Unique keys permitted per table.

Q91. Foreign Key vs Composite Key?

- **Foreign Key:** A column or group of columns in one table that references the Primary Key of another table, enforcing referential data integrity.
- **Composite Key:** A primary or unique key made up of **two or more columns** combined to uniquely identify a record (e.g. `OrderId + ProductId` in OrderDetails table).